ЗАЩИТА ПО ЗАДАНИЯМ ОТЧЁТА

MCP сервер для игрового движка Godot

Павлов С.К. и Трещев О.Д., ИДБ-25-15

Преподаватели: Лобанов О.А., Кайкова И.В.

Это сценарий разговора по отчёту, без привязки к презентации. Открывайте указанный раздел и рисунок. Основные реплики займут около 10 минут без вопросов. Блоки «Вопрос — ответ» нужны для подготовки; читать их подряд на защите не нужно. Если преподаватель сразу спрашивает схему, переходите к ней. Оба участника должны понимать все схемы.

ВСТУПЛЕНИЕ — СТЕПАН

Показать титульный лист и оглавление.

Наша тема — MCP сервер для игрового движка Godot. Мы рассматриваем разработку игр как цифровое производство: на входе код, сцены и ресурсы, затем выполняются изменение и проверка, на выходе — изменённый проект и результаты контроля. Наш продукт даёт клиенту инструменты для выполнения части этих операций.

В первом задании мы определили функционал и участников, во втором построили IDEF0 и IDEF3, в третьем описали организацию и бизнес-процессы средствами ARIS. Техническую часть проверили на работающем прототипе.

ЗАДАНИЕ 1. ТЕМА, ЦЕЛЬ И ФУНКЦИОНАЛ — ОЛЕГ

Открыть разделы 1 и 2.

При работе с ИИ разработчик часто вручную переносит информацию между чатом и Godot. Например, агент написал код, но результат запуска нужно отдельно проверить и пересказать. Мы хотим, чтобы клиент получал диагностические данные непосредственно через инструменты сервера.

Цель к защите — получить повторяемый цикл изменения и проверки проекта. Критерии конкретные: создать сцену, добавить узел, прочитать его в структуре, запустить исправный проект и обнаружить намеренно сломанный скрипт. Мы не заявляем измеренное ускорение: для этого нужен отдельный сравнительный эксперимент.

Объект исследования — процесс разработки игр в Godot с участием ИИ. Предмет — способы связи клиента с движком и передачи результатов проверки. Объект шире, а предмет — та часть процесса, которую мы изменяем.

Пользователи — самостоятельные разработчики, небольшие команды и студенты. Практическая новизна нашей версии — два расширения выбранной базы и повторяемый проверочный сценарий. Саму идею MCP для Godot мы не изобретали.

Вопрос: Почему цель не «создать сервер»?

Ответ: Создание — действие. Нужный эффект — получать проверяемый результат изменения проекта без ручного переноса диагностики. Достижение цели оцениваем по заданным сценариям.

Вопрос: Где здесь ИИ?

Ответ: В подключённом клиенте и его модели. Сам MCP сервер выполняет структурированные вызовы; его может вызывать и обычная программа.

ЗАДАНИЕ 1. АНАЛОГИ И СОБСТВЕННЫЙ ВКЛАД — СТЕПАН

Открыть раздел 3 и сравнительную таблицу; затем таблицу возможностей раздела 2.

Мы взяли открытый Coding-Solo/godot-mcp. Сравнение с выбранной базой охватывает пять критериев: подключение, операции со сценой, проверку проекта, контроль параметров расширений и воспроизводимость приёмки. Рассмотрели также ee0pdt и mkdevkit, использующие интеграцию с редактором. Одинаковых локальных испытаний всех аналогов мы не проводили.

В базе было четырнадцать инструментов. Мы добавили два. godot_inspect_scene читает имена, типы и пути узлов сохранённой сцены. godot_validate_project импортирует проект, запускает его на ограниченное число кадров и возвращает статус и логи. Ещё подготовлены тест, демонстрация и документация. Исходные авторство и лицензия сохранены.

Вопрос: Что именно сделали вы?

Ответ: Два расширения, интеграционный тест, демонстрационный сценарий и документацию. Исходный сервер и четырнадцать инструментов не выдаём за собственную разработку. Изменения выполнялись с помощью ИИ и выделены в CHANGES.md.

Вопрос: Почему не с нуля?

Ответ: База позволила сосредоточиться на проверке и обратной связи. Мы явно указываем происхождение кода и границы своего вклада.

Вопрос: Что такое сохранённая сцена?

Ответ: Данные файла .tscn. Узлы, появившиеся только во время игры, могут отсутствовать. Это не просмотр живого дерева работающей игры.

ЗАДАНИЕ 1. UML — СТЕПАН

Открыть раздел 6, рисунок 1. Показать рамку, участников, овалы.

Рамка задаёт границы MCP сервера. Внутри — создание и изменение сцены, чтение структуры, запуск проверки, получение логов и статуса. Снаружи — разработчик и MCP-клиент.

Разработчик инициирует действие, а технический вызов выполняет клиент. Связь с разработчиком показывает его участие в сценарии, а не прямое соединение с сервером. Овалы отвечают на вопрос «что можно сделать». Порядка действий эта схема не задаёт: например, структуру можно прочитать отдельно от запуска проверки.

Вопрос: Почему нет include и extend?

Ответ: Здесь показаны самостоятельные сценарии. Обязательное включение одного в другой не моделируется. Эти связи не нужны только ради наличия на схеме.

Вопрос: Чем отличается от алгоритма?

Ответ: Use Case показывает возможности и участников. Порядок и ветвления показаны в IDEF3.

Вопрос: Где Godot?

Ответ: В этой обзорной схеме выделены пользовательские участники. Исполняющая система показана в архитектуре; при детализации UML Godot можно добавить как вторичного участника.

ЗАДАНИЕ 1. ОРГАНИЗАЦИЯ — ОЛЕГ

Показать распределение компетенций в разделе 6 и рисунок 7 в разделе 11.

Нас двое, поэтому мы не изображаем вымышленные отделы. Степан отвечает за интеграцию, конфигурации, тесты и документацию. Я — за сценарии Godot, демонстрацию и критерии приёмки. Требования и развитие продукта обсуждаем совместно.

Организационная схема показывает ответственность людей. В ARIS используем это же распределение для связи процессов с исполнителями. Это план распределения работ, а не утверждение, что каждый лично написал определённое количество строк.

Вопрос: Кто руководитель?

Ответ: Решения согласуются двумя участниками. Верхний блок обозначает проект, а не должность отдельного руководителя.

Вопрос: Зачем модель двум людям?

Ответ: Чтобы определить, кто готовит реализацию, кто проверяет сценарий и кто отвечает за документацию.

ЗАДАНИЕ 1. СТРУКТУРА И ПОТОКИ — СТЕПАН

Открыть раздел 7, рисунки 2 и 3.

Рисунок 2 показывает компоненты. Разработчик задаёт требования клиенту. Клиент работает с моделью и вызывает MCP. Сервер на TypeScript работает в Node.js и запускает Godot через командную строку и служебные GDScript. Изменения сохраняются в файлах проекта.

Рисунок 3 показывает передаваемую информацию. Пользователь отправляет задачу. Клиент передаёт название инструмента и параметры в JSON. Сервер вызывает Godot. Движок работает с ресурсами и возвращает диагностические данные. Клиент использует ответ для следующего действия или объяснения пользователю.

Состав системы и движение данных — разные вопросы, поэтому нужны обе схемы.

Вопрос: Где хранится проект?

Ответ: В локальной папке пользователя. Серверу передаётся путь к ней.

Вопрос: Все данные остаются локально?

Ответ: Сам сервер работает локально, но ИИ-клиент может отправлять контекст внешней модели. Это зависит от клиента.

Вопрос: Что такое stdio?

Ответ: Стандартные потоки ввода и вывода процесса. Клиент запускает сервер и обменивается с ним сообщениями через эти потоки.

Вопрос: Управляете открытым редактором?

Ответ: Текущая реализация в основном запускает отдельные процессы Godot. Постоянного плагина для живого управления открытым редактором в наших расширениях нет.

ЗАДАНИЕ 2. КОНТЕКСТ IDEF0 — СТЕПАН

Открыть раздел 8, рисунок 4. Показать четыре стороны блока.

IDEF0 описывает функцию и необходимые для неё условия. Общая функция A0 — автоматизировать изменение и проверку проекта. Контекстная диаграмма называется A-0: система здесь представлена одним блоком.

Слева вход — исходный проект и задание. Сверху управление — требования, лицензии, версии и лимиты. Снизу механизмы — разработчик, клиент, MCP, Godot и компьютер. Справа выход — изменённый проект и отчёт.

Задание содержит описание изменения, а управление — критерии и ограничения. Например, «добавить надпись» — задание, а «не менять другие сцены» — ограничение.

Вопрос: Почему Godot снизу?

Ответ: Это средство выполнения. Мы преобразуем проект, а не сам движок.

Вопрос: Почему требования сверху?

Ответ: Они задают правила и критерии допустимого результата.

Вопрос: Чем A-0 отличается от A0?

Ответ: A-0 — контекстная диаграмма. На ней единственная функция A0. Диаграмма декомпозиции A0 раскрывает эту функцию в дочерние функции.

Вопрос: Стрелки показывают время?

Ответ: Нет. Они показывают входы, выходы, управление и механизмы. Сценарий по времени описываем отдельно.

ЗАДАНИЕ 2. ДЕКОМПОЗИЦИЯ IDEF0 — СТЕПАН

Открыть раздел 9, рисунок 5 и таблицу.

Мы разбили функцию на четыре части. A1 — уточнить задание. A2 — изменить проект. A3 — проверить результат. A4 — принять изменение. Так мы отделяем создание файлов от проверки и решения о готовности.

Уточнённое задание используется для изменения проекта. Изменённый проект поступает в проверку, диагностический результат — в приёмку. Общие правила управляют функциями, инструменты и участники обеспечивают выполнение. Подробные входы и результаты дополнительно указаны в таблице.

Даже успешный запуск не означает, что результат соответствует замыслу. Например, игра запускается, но надпись расположена неправильно. Это выявляется при ручной приёмке.

Вопрос: Почему четыре блока?

Ответ: Это самостоятельные функции с разными результатами. Для нашего обзорного уровня такое разделение позволяет понять ответственность и контроль.

Вопрос: Кто исправляет ошибку?

Ответ: Разработчик или ИИ-клиент. MCP возвращает диагностику, но не выбирает исправление самостоятельно.

ЗАДАНИЕ 2. IDEF3 — ОЛЕГ

Открыть раздел 10, рисунок 6. Провести сначала по успешному пути, затем по ошибочному.

Здесь показан порядок действий. После изменения проекта проверяем запуск. Если проверка успешна, переходим к приёмке. Если обнаружена ошибка, разбираем причину и решаем, допустима ли ещё одна попытка.

Если исправление понятно и укладывается в ограничения, корректируем проект и возвращаемся к проверке. Если причина неясна или лимит исчерпан, передаём проблему разработчику. Решение о повторе принимает клиент или человек, а не инструмент проверки.

Прямоугольник — единица работы, стрелка — порядок, X — исключающее ветвление или слияние. XOR означает выбор одной альтернативы. Успешная проверка ведёт к приёмке человеком, а не заменяет её.

Вопрос: Почему XOR, а не AND?

Ответ: Выбирается один исход: успех либо ошибка. AND означал бы выполнение всех веток.

Вопрос: Чем отличается от IDEF0?

Ответ: IDEF0 показывает функции и ресурсы, IDEF3 — сценарий и альтернативные продолжения.

Вопрос: Что останавливает повторы?

Ответ: Правила клиента: число попыток, время и допустимый объём изменений. Ограничение кадров и таймаут самого инструмента — дополнительный, отдельный механизм.

ЗАДАНИЕ 3. ОРГАНИЗАЦИОННАЯ МОДЕЛЬ И ДЕРЕВО ARIS — ОЛЕГ

Открыть раздел 11, рисунки 7 и 8.

Организационная модель отвечает на вопрос «кто работает». Дерево процессов отвечает на вопрос «из каких направлений состоит деятельность».

В корне — предоставить средство производства игр. Ниже — разработать MCP, поставить продукт, сопровождать клиента и развивать продажи. Разработка включает требования, инструменты и тесты. Поставка — пакет, инструкцию, установку и первую операцию. Сопровождение — диагностику, обновления и поддержку. Продажи — демонстрации, привлечение пользователей и проверку платного предложения.

Это целевая модель. Реализация, тест и инструкция уже есть; продажи и постоянная платная поддержка пока планируются. Дерево не задаёт порядок во времени: направления могут выполняться параллельно.

Вопрос: Почему есть продажи, которых ещё нет?

Ответ: Мы проектируем полный процесс продукта и отдельно отмечаем готовую часть и планы.

Вопрос: Почему дерево не совпадает с IDEF0?

Ответ: Разные границы: IDEF0 описывает изменение проекта клиента, а дерево ARIS — жизненный цикл нашего инструмента. Это не одна и та же декомпозиция.

ЗАДАНИЕ 3. СОБЫТИЙНАЯ ЦЕПОЧКА ARIS — ОЛЕГ

Открыть раздел 12, рисунок 9. Читать сверху вниз.

Начальное событие — задание получено. Следующая функция — выполнить изменение и проверку. XOR разделяет два результата: проверка пройдена или обнаружена ошибка.

При успехе разработчик просматривает и принимает изменение. При ошибке разбирается проблема и создаётся задача на доработку. Ветки объединяются, статус фиксируется в отчёте, и итерация завершается.

Шестиугольник — событие, то есть наступившее состояние. Прямоугольник — функция, то есть действие. Дополнительные обозначения связывают функции с исполнителями и данными: технические операции выполняют клиент и MCP, при фиксации результата используются логи и список изменений.

Здесь показана одна завершённая итерация. Доработка начинается следующей итерацией. IDEF3 подробнее показывает возможный повтор проверки, а эта схема — фиксацию исхода одной итерации.

Вопрос: Разница события и функции?

Ответ: «Ошибка обнаружена» — состояние; «разобрать ошибку» — действие.

Вопрос: Почему слияние XOR?

Ответ: Фиксируем один из альтернативных результатов. Обе ветки одновременно не нужны.

Вопрос: Кто принимает результат?

Ответ: Разработчик. Сервер сообщает технический статус, на основании которого человек принимает решение.

ОТЧЁТНОСТЬ И МАКЕТ — ОБА УЧАСТНИКА

Открыть раздел 13, Godot и файлы результатов.

Степан: В src/extensions.ts находятся два расширения, в tests/integration.test.mjs — тест. Он подключается к реальному серверу, получает шестнадцать инструментов, создаёт сцену и узел, читает дерево и запускает проверку.

Олег: Открываем demo-defense/project.godot и main.tscn. Узел Title добавлен через MCP. Это обычный редактируемый проект. В mcp-trace.json видны вызовы и ответы: структура сцены, импорт, запуск и их статусы.

Степан: В evidence/acceptance.json сохранена проверка успешного и ошибочного сценариев. Исправный проект проходит запуск; неверные параметры и выход за папку отклоняются; намеренно неправильный GDScript возвращает ошибку. Повторный тест прошёл на Windows с Godot 4.7.1.

Олег: Демонстрацию выполняет программный MCP-клиент. Мы показываем протокол и инструменты, а не выдаём заранее заданный сценарий за диалог с ИИ. Интерактивное подключение конкретных ИИ-клиентов проверяется отдельно.

Вопрос: Реализованы все процессы схем?

Ответ: Прототип покрывает технические операции. Уточнение требований, выбор исправлений и приёмка остаются за человеком и клиентом; коммерческие процессы относятся к развитию.

Вопрос: После проверки в игре нет ошибок?

Ответ: Нет. Импорт и короткий запуск не проверяют все игровые ситуации, графику и качество геймплея.

Вопрос: Можно безопасно проверять любой чужой проект?

Ответ: Нет. Godot выполняет код проекта с правами пользователя. Мы используем отдельный доверенный проект.

ЕСЛИ СПРОСЯТ БИЗНЕС-МОДЕЛЬ — ОЛЕГ

Открыть разделы 4–5 только при вопросе преподавателя.

Гипотеза — открытое ядро и платное сопровождение. В учебном расчёте цена 990 рублей, переменные расходы 190 рублей, вклад пользователя 800 рублей. При постоянных расходах 50 тысяч рублей покрытие расходов достигается при 63 пользователях. Налоги, возвраты и полная стоимость разработки здесь не учтены. Продаж пока нет.

Вопрос: За что платить при бесплатных аналогах?

Ответ: Возможно, за установку, настройку и сопровождение. Если пилот не подтвердит такую потребность, гипотезу нужно менять.

ЗАВЕРШЕНИЕ — СТЕПАН

Мы определили функционал и ответственность, описали устройство системы и построили модели функций, сценариев и событий. Техническую часть подтвердили прототипом и тестом. Следующий шаг — проверка в реальных ИИ-клиентах и измерение пользы на одинаковых задачах.

ПАМЯТКА

Открыть рисунки 1–9 и найти каждый блок до защиты. Запомнить четыре стороны IDEF0, XOR, разницу события и функции и разницу сохранённой сцены и живого дерева.

Перед защитой выполнить npm test с заданным GODOT_PATH. Для нового прогона: node scripts/demo.mjs demo-live; папка demo-live не должна существовать. Подготовить Godot, mcp-trace.json и acceptance.json заранее.

Если живой запуск не удался, показать сохранённый результат и прямо назвать его предварительным прогоном. Не утверждать, что все инструменты написаны самостоятельно, клиенты полностью проверены, все ошибки исключены или продажи состоялись.